

Building Intelligent Troubleshooting Agents

Module 1

LLM fine-tuning for task-specific adaptation

Module 2

Fundamentals of AI agents

Module 3

Natural language processing for troubleshooting

Module 4

Implementing the troubleshooting agent

Module 5

Testing and optimizing the agent

Designing test cases

Video • 8 min

Explanation of test case design

Reading • 10 min

Practice activity: Designing test cases for ML systems

Reading • 10 min

Reflection: Designing test cases for ML systems

Practice Assignment • 1 min

Walkthrough: Designing test cases for ML systems (Optional)

Video • 5 min

Hear from an expert: Accounting for cultural, language, and contextual nuances

Video • 4 min

Optimizing response time and accuracy

Reading • 10 min

Explanation of optimization techniques

Video • 6 min

Practice activity: Implementing optimization techniques

Reading • 10 min

Reflection: Implementing optimization techniques

Practice Assignment • 3 min

Walkthrough: Implementing optimization techniques (Optional)

Video • 7 min

Evaluating agent effectiveness

Reading • 10 min

Practice activity: Evaluating agent effectiveness

Reading • 10 min

Reflection: Evaluating agent effectiveness

Practice Assignment • 3 min

Walkthrough: Evaluating agent effectiveness (Optional)

Video • 5 min

Hear from an expert: Designing with the end user in mind

Video • 2 min

Summary: Testing and optimizing the agent

Video • 6 min

Practice activity: Testing and optimizing the agent

Reading • 10 min

Reflection: Testing and optimizing the agent

Practice Assignment • 3 min

Walkthrough: Testing and optimizing the ML agent (Optional)

Video • 4 min

Graded quiz: Testing and optimizing the agent

Graded Assignment • 10 min

Hear from an expert: Resolving unexpected issues during implementation

Video • 5 min

Course summary

Video • 6 min

Course assignment: Producing a troubleshooting agent

Reading • 10 min

Reflection: Producing a troubleshooting agent

Practice Assignment • 1 min

Walkthrough: Producing a troubleshooting agent (Optional)

Video • 4 min

Course assignment: Drafting the technical report

Peer graded Assignment • 10-30m

Course assignment: Drafting the technical report

Review Your Draft

Congratulations on completing the course!

Video • 3 min

Practice activity: Designing test cases for ML systems

Introduction

In this activity, you will design and implement test cases to ensure that your machine learning (ML) model performs effectively across typical, edge, and error scenarios. The goal is to create a robust suite of test cases that validate the model's behavior in a variety of conditions, ensuring its reliability, accuracy, and resilience.

By the end of this activity, you will:

- Design test cases for typical scenarios based on representative input data.
- Develop test cases for edge scenarios, focusing on extreme, unusual, or unexpected inputs.
- Automate test execution to validate the model after updates or when new data is introduced.

Step-by-step process to design test cases for ML systems

Create a new Jupyter notebook. Make sure you have the appropriate Python 3.8 Azure ML kernel selected.

The remaining of this reading will guide you through the following steps:

- Step 1: Set up the environment
- Step 2: Define test cases for typical scenarios
- Step 3: Design test cases for edge scenarios
- Step 4: Create error-handling test cases
- Step 5: Automate test case execution
- Step 6: Evaluate the effectiveness of your test cases

Step 1: Set up the environment

Begin by setting up your Python environment. Make sure you have a testing framework like pytest or unittest installed to support automated testing.

Setup commands

```
1 pip install pytest
2 !pip install pytest
```

Explanation

These libraries will help you automate test case execution and ensure that the test cases can be run efficiently every time your model is updated.

Let's create a model.

```
1 import pytest
2 import numpy as np
3 from sklearn.datasets import load_iris
4 from sklearn.tree import DecisionTreeClassifier
5 import numpy as np
6
7 # Load the dataset
8 iris = load_iris()
9 X, y = iris.data, iris.target
10
11 # Set up the model
12 model = DecisionTreeClassifier()
13 model.fit(X, y)
```

Here we've used the Iris dataset and a decision tree classifier to create a decision tree classifier model.

Step 2: Define test cases for typical scenarios

Start by defining test cases for typical scenarios. These scenarios represent the normal input data your model will frequently encounter during regular operation.

Instructions

- Identify representative input data from your dataset.
- Define the expected output for each test case based on your model's behavior.
- Create a set of test cases that cover the most common use cases.

Code example

```
1 def test_typical_case():
2     input_data = np.array([[4.5, 1.2, 1.3, 0.3]]) # Example input for a flower classification model
3     expected_output = 0 # Expected output for typical case (Setosa class index)
4     result = model.predict(input_data)[0]
5     assert result == expected_output, f"Expected {expected_output}, but got {result}"
```

Explanation

This test case checks whether the model correctly classifies a typical input, ensuring it handles common situations well.

Step 3: Design test cases for edge scenarios

Next, create test cases that challenge the model with edge scenarios. These tests use data that falls outside the normal range and include extreme or unusual values.

Instructions

- Identify edge cases with unusually high or low values.
- Test how the model responds to these edge cases and ensure robustness.
- Define the expected output or error handling for each edge case.

Code example

```
1 def test_edge_case_extreme_value():
2     input_data = np.array([[1000, 1000, 1000, 1000]]) # Extreme values for flower classification
3     try:
4         model.predict(input_data)
5     except ValueError:
6         assert True # The model should raise a ValueError for extreme inputs
7     else:
8         assert False, "Expected ValueError for extreme values, but no error was raised"
```

Explanation

Edge case testing ensures that the model doesn't produce incorrect results or crash when encountering extreme or unusual inputs.

Step 4: Create error-handling test cases

Now create test cases to verify the model's ability to handle invalid inputs. Error-handling tests check whether the model behaves correctly when it receives flawed or incomplete data.

Instructions

- Create test cases with missing, null, or malformed data.
- Ensure that the model handles these cases appropriately, either by returning an error message or handling the input gracefully.

Code example

```
1 def test_error_handling_missing_values():
2     input_data = np.array([[None, None, None, None]]) # Missing values in input
3     try:
4         model.predict(input_data)
5     except ValueError:
6         assert True # The model should raise a ValueError for missing inputs
7     else:
8         assert False, "Expected ValueError for missing values, but no error was raised"
```

Explanation

This test case checks how the model handles missing data, ensuring it doesn't fail or produce invalid results when inputs are incomplete.

Step 5: Automate test case execution

After defining your test cases, automate the testing process using pytest or unittest. Automation ensures that tests are run every time the model is updated, preventing errors from slipping through.

Instructions

- Write automated tests for each of your test cases.
- Use a framework such as pytest to automatically execute all tests.
- Set up continuous testing to validate the model after updates or new data are introduced.

Code example (with pytest)

```
1 # Run tests using pytest
2 pytest -xv
```

Explanation

Automating test execution ensures that all test cases are checked after each update, helping to catch potential issues before deployment.

Step 6: Evaluate the effectiveness of your test cases

Finally, evaluate your test cases to ensure they comprehensively cover typical, edge, and error scenarios. Refine them based on the results to improve coverage and accuracy.

Instructions

- Review the test results to ensure the model passes all cases.
- Check whether the model behaves as expected in edge and error scenarios.
- Improve the test cases if any failures or unexpected behaviors are identified.

Explanation

Evaluating the effectiveness of your test cases helps ensure that they thoroughly test the model's behavior and identify any areas where the model might fail.

Conclusion

In this activity, you designed and implemented test cases for an ML model, covering typical, edge, and error scenarios. By automating the test execution process, you ensure that your model remains robust, reliable, and well-suited for real-world applications, even as it is updated or exposed to new data.

Work is completed

Like Dislike Report an issue

coach

Hi, Venkatesh!

How can I help?

Explain this topic in simple terms

Give me a summary

Give me practice questions

Give me real-life examples

Ask me anything

↓

Coach is powered by AI, so check for mistakes and don't share sensitive info. Your data will be used in accordance with OpenAI's Privacy Notice

Go to next item →